# INFORME FINAL

## Instalador de Aplicación y Ciclo de Pruebas

*Sistema de Gestión de Restaurante — MVC PyQt6*

| | |
|---|---|
| **Actividad:** | Instalador + Pruebas del Sistema |
| **Sistema:** | Gestión de Restaurante |
| **Tecnología:** | Python 3.8+ · CustomTkinter · SQLAlchemy · SQLite |
| **Herramienta de instalación:** | PyInstaller 6.x / Inno Setup 6 |
| **Framework de pruebas:** | pytest · unittest · pytest-cov |
| **Repositorio:** | GitHub — Alexander200318 |
| **Entregable:** | Link de repositorio GitHub con instalador |
| **Fecha:** | 2026 |
| **Integrantes** | Adriana Cornejo U, Ismael Loja, Victor Mendez, Alfonso Chevez |

## 1. Introducción

El presente informe documenta la fase final del desarrollo del Sistema de Gestión de Restaurante, correspondiente a la creación del instalador ejecutable de la aplicación y la ejecución del ciclo completo de pruebas de software. Esta etapa es fundamental para garantizar que el sistema pueda ser distribuido, instalado y utilizado correctamente por los usuarios finales en entornos reales.

El informe detalla el proceso de construcción del instalador mediante PyInstaller y la generación del asistente de instalación con Inno Setup, así como el diseño y ejecución de pruebas unitarias, de integración y de interfaz gráfica utilizando las librerías del ciclo de pruebas de Python.

El entregable principal de esta actividad es el link del repositorio de GitHub que contiene tanto el código fuente actualizado como el instalador generado, los scripts de prueba y los reportes de resultados.

# 2. Objetivos

## 2.1 Objetivo General

Desarrollar el instalador ejecutable del Sistema de Gestión de Restaurante, subirlo al repositorio de GitHub y ejecutar el ciclo completo de pruebas del software, documentando los resultados obtenidos.

## 2.2 Objetivos Específicos

- Generar el ejecutable de la aplicación mediante PyInstaller, empaquetando todas las dependencias.
- Crear un asistente de instalación profesional (.exe) con Inno Setup para distribución en Windows.
- Subir el instalador y scripts de prueba al repositorio de GitHub del proyecto.
- Implementar pruebas unitarias con pytest y unittest para validar la lógica de negocio.
- Ejecutar pruebas de integración verificando la comunicación entre controladores y base de datos.
- Documentar los casos de prueba, resultados obtenidos y reporte de cobertura del código.

# 3. Desarrollo del Instalador

> Herramientas utilizadas: PyInstaller 6.x (empaquetado Python → .exe) + Inno Setup 6 (asistente de instalación profesional para Windows)

## 3.1 Herramientas de Empaquetado

| Herramienta | Versión | Función |
|---|---|---|
| PyInstaller | 6.x | Convierte el proyecto Python en un ejecutable standalone (.exe) incluyendo todas las dependencias y librerías. |
| Inno Setup | 6.x | Crea el asistente de instalación gráfico (.exe installer) con opciones de directorio, accesos directos y desinstalador. |
| auto-py-to-exe | Latest | Interfaz gráfica opcional sobre PyInstaller para configurar el empaquetado de forma visual. |

| | | |
|---|---|---|
| UPX | 4.x | Compresor de ejecutables (opcional) para reducir el tamaño del archivo final generado. |

## 3.2 Proceso de Empaquetado con PyInstaller

El proceso de generación del ejecutable sigue los siguientes pasos:

1. **Instalación de PyInstaller:** Se instala la herramienta en el entorno virtual del proyecto.
2. **Generación del archivo. spec:** PyInstaller analiza el proyecto y genera un archivo de configuración. spec con todas las dependencias detectadas.
3. **Configuración de recursos:** Se ajusta el .spec para incluir archivos adicionales: base de datos, íconos, imágenes e assets del sistema.
4. **Construcción del ejecutable:** Se ejecuta el comando de empaquetado generando la carpeta dist/ con el ejecutable final.
5. **Verificación del ejecutable:** Se prueba el .exe en un entorno limpio (sin Python instalado) para validar que funciona correctamente de forma independiente.

Comandos utilizados durante el proceso de empaquetado:

```
# 1. Instalar PyInstaller en el entorno virtual
pip install pyinstaller

# 2. Generar ejecutable básico (modo onefile)
pyinstaller --onefile --windowed --name RestauranteApp main.py

# 3. Generar con ícono personalizado
pyinstaller --onefile --windowed --icon=assets/icon.ico --name RestauranteApp
main.py

# 4. Usando archivo .spec para configuración avanzada
pyinstaller RestauranteApp.spec

# 5. Resultado → dist/RestauranteApp.exe
```

## 3.3 Configuración del Archivo .spec

El archivo RestauranteApp.spec permite configurar el empaquetado de forma detallada, incluyendo todos los archivos necesarios para el correcto funcionamiento:

```
# RestauranteApp.spec
a = Analysis(
    ['main.py'],
```

```
    pathex=['.'],
    binaries=[],
    datas=[
        ('assets/',         'assets/'),       # Íconos e imágenes
        ('database/',       'database/'),     # Módulos de BD
        ('restaurante.db',  '.'),             # Base de datos SQLite
    ],
    hiddenimports=[
        'customtkinter',
        'sqlalchemy',
        'PIL',
        'matplotlib',
        'reportlab',
    ],
)
exe = EXE(pyz, a.scripts, name='RestauranteApp',
          debug=False, console=False, icon='assets/icon.ico')
```

## 3.4 Creación del Instalador con Inno Setup

Una vez generado el ejecutable con PyInstaller, se utiliza Inno Setup para crear un instalador profesional con asistente gráfico. El instalador incluye:

- Asistente paso a paso (wizard) para guiar al usuario durante la instalación.
- Selección del directorio de instalación personalizable.
- Creación automática de acceso directo en el Escritorio y menú Inicio de Windows.
- Inclusión de la base de datos SQLite inicial con datos de prueba (seed_data).
- Desinstalador automático generado y registrado en el Panel de Control de Windows.
- Verificación de versión mínima del sistema operativo (Windows 10/11).

```
; RestauranteApp_Setup.iss — Script de Inno Setup
[Setup]
AppName=Sistema de Gestión de Restaurante
AppVersion=1.0.0
AppPublisher=Universidad — Desarrollo de Software
DefaultDirName={autopf}\RestauranteApp
DefaultGroupName=RestauranteApp
OutputBaseFilename=RestauranteApp_Installer_v1.0
Compression=lzma
SolidCompression=yes
WizardStyle=modern
MinVersion=10.0

[Files]
Source: "dist\RestauranteApp.exe"; DestDir: "{app}"
Source: "restaurante.db";            DestDir: "{app}"
Source: "assets\*";                 DestDir: "{app}\assets"

[Icons]
```

```
Name: "{autoprograms}\RestauranteApp"; Filename: "{app}\RestauranteApp.exe"
Name: "{autodesktop}\RestauranteApp";  Filename: "{app}\RestauranteApp.exe"
```

### 3.5 Estructura del Repositorio tras el Instalador

Una vez generado el instalador, se actualiza el repositorio de GitHub con los nuevos archivos:

```
restaurante_mvc_pyqt6/
├── src/                          # Código fuente del proyecto
│   ├── main.py
│   ├── models/
│   ├── views/
│   ├── controllers/
│   └── database/
├── installer/                    # Archivos del instalador
│   ├── RestauranteApp.spec       # Configuración PyInstaller
│   ├── RestauranteApp_Setup.iss  # Script Inno Setup
│   └── RestauranteApp_Installer_v1.0.exe  # Instalador final
├── tests/                        # Scripts de prueba
│   ├── test_models.py
│   ├── test_controllers.py
│   ├── test_database.py
│   └── test_integration.py
├── reports/                      # Reportes de pruebas
│   └── coverage_report/
├── requirements.txt
└── README.md
```

## 4. Librerías para el Ciclo de Pruebas

El ciclo de pruebas del Sistema de Restaurante utiliza un conjunto de librerías especializadas de Python para garantizar la calidad, correctitud y robustez del software en todas sus capas.

### 4.1 Tabla de Librerías de Prueba

| Librería | Versión | Tipo de Prueba | Propósito en el Proyecto |
|---|---|---|---|
| pytest | 8.x | **Unitaria** | Framework principal de pruebas. Permite escribir y ejecutar tests de forma simple con detección automática de archivos test_*.py. |
| unittest | Stdlib | **Unitaria** | Módulo estándar de Python para pruebas unitarias. Usado para TestCase de modelos, controladores y validadores del sistema. |
| pytest-cov | 4.x | **Cobertura** | Plugin de pytest para medir la cobertura de código (code coverage). Genera reportes HTML y XML del porcentaje de código probado. |
| coverage.py | 7.x | **Cobertura** | Librería base de medición de cobertura utilizada por pytest-cov. Permite identificar líneas de código no ejecutadas durante las pruebas. |

| | | | |
|---|---|---|---|
| pytest-mock | 3.x | Unitaria | Plugin para crear objetos Mock y Patch en pruebas. Permite simular dependencias externas como base de datos y APIs sin ejecutarlas realmente. |
| unittest.mock | Stdlib | Unitaria | Módulo estándar de Python para mocking. Usado para simular llamadas a SQLAlchemy y CustomTkinter en pruebas unitarias aisladas. |
| SQLAlchemy Testing | 2.x | Integración | Utilidades de SQLAlchemy para pruebas con base de datos en memoria (SQLite :memory:), permitiendo pruebas de integración sin afectar la BD real. |
| pytest-qt / tkinter | — | Interfaz (GUI) | Permite simular eventos de interfaz gráfica (clicks, escritura) y verificar el estado de widgets en pruebas automatizadas de la GUI. |
| behave | 1.2.x | Comportamiento (BDD) | Framework de pruebas BDD (Behavior-Driven Development). Permite escribir pruebas en lenguaje natural (Gherkin) para escenarios de negocio. |
| pytest-html | 4.x | Cobertura | Genera reportes HTML visuales con el resultado de cada prueba, estadísticas, duración y detalles de fallos para entrega o revisión. |

## 4.2 Descripción Detallada de Librerías

### pytest — Framework Principal de Pruebas

pytest es el framework más utilizado en el ecosistema Python para escribir y ejecutar pruebas automatizadas. Su sintaxis simple basada en funciones (sin necesidad de clases) permite crear tests de forma rápida y legible. Detecta automáticamente los archivos test_*.py y las funciones que comienzan con test_.

```
# Instalación
pip install pytest

# Ejecución de todas las pruebas
pytest tests/

# Ejecución con reporte detallado
pytest tests/ -v

# Ejemplo de prueba unitaria con pytest
def test_calcular_propina():
    subtotal = 100.00
    propina  = subtotal * 0.10
    assert propina == 10.00
```

### unittest — Módulo Estándar de Pruebas

unittest es el framework de pruebas incluido en la biblioteca estándar de Python, basado en el patrón xUnit. Se utiliza en el proyecto para pruebas orientadas a clases con métodos setUp() y tearDown() que preparan y limpian el entorno de cada prueba.

```
import unittest
from models.pedidos import PedidoModel

class TestPedidoModel(unittest.TestCase):
    def setUp(self):
        # Prepara entorno de prueba con BD en memoria
        self.model = PedidoModel(db_url='sqlite:///:memory:')

    def test_crear_pedido_exitoso(self):
        success, pedido, msg = self.model.crear(mesa_id=1)
        self.assertTrue(success)
        self.assertIsNotNone(pedido)

    def tearDown(self):
        # Limpia la sesión de prueba
        self.model.session.close()
```

### pytest-cov y coverage.py — Cobertura de Código

pytest-cov es un plugin de pytest que integra coverage.py para medir qué porcentaje del código fuente es ejecutado durante las pruebas. El reporte identifica las líneas no cubiertas, permitiendo mejorar la calidad del conjunto de pruebas. Se busca alcanzar un mínimo del 80% de cobertura.

```
# Instalación
pip install pytest-cov coverage

# Ejecutar pruebas con cobertura
pytest tests/ --cov=. --cov-report=html --cov-report=term

# Resultado en terminal:
# ----------- coverage: platform win32 -----------
# Name                        Stmts   Miss  Cover
# controllers/pagos.py           45      4    91%
# controllers/pedidos.py         62      8    87%
# models/mesas.py                38      2    95%
# TOTAL                         380     32    92%
```

### pytest-mock y unittest.mock — Simulación de Dependencias

Los módulos de mocking permiten reemplazar objetos reales (base de datos, APIs, widgets) con objetos simulados (mocks) durante las pruebas unitarias. Esto garantiza que cada unidad se pruebe de forma aislada, sin depender de la base de datos real ni de la interfaz gráfica.

```
from unittest.mock import patch, MagicMock
import pytest

def test_pago_registrado_correctamente(mocker):
    # Simula la sesión de SQLAlchemy sin tocar la BD real
    mock_session = mocker.patch('controllers.pagos.get_session')
    mock_session.return_value.__enter__ = MagicMock()

    from controllers.pagos_controller import PagosController
    ctrl = PagosController()
    success, _, msg = ctrl.registrar_pago(pedido_id=1, monto=50.00,
                                          metodo='EFECTIVO')
    assert success == True
    assert 'exitoso' in msg.lower()
```

### SQLAlchemy Testing — Pruebas de Integración con BD en Memoria

Para las pruebas de integración, se utiliza SQLite en modo :memory: (base de datos en RAM) junto con las utilidades de SQLAlchemy para crear y destruir tablas entre cada prueba. Esto permite probar la interacción real entre controladores y base de datos sin afectar los datos de producción.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from database.models import Base

# Motor en memoria para pruebas (se destruye al finalizar)
engine  = create_engine('sqlite:///:memory:')
Session = sessionmaker(bind=engine)

def setup_test_db():
    Base.metadata.create_all(engine)   # Crea todas las tablas
    return Session()

def teardown_test_db(session):
    session.close()
    Base.metadata.drop_all(engine)     # Elimina todas las tablas
```

### behave — Pruebas de Comportamiento (BDD)

behave permite definir pruebas en lenguaje natural mediante la sintaxis Gherkin (Given / When / Then), facilitando que tanto desarrolladores como stakeholders comprendan los escenarios de prueba. Se utiliza para validar los flujos principales del restaurante desde la perspectiva del negocio.

```
# features/pedidos.feature
Feature: Gestión de Pedidos
```

```
  Scenario: Crear un pedido exitosamente
    Given que existe una mesa con número 5 en estado LIBRE
    When el mesero registra un pedido con 2 platos
    Then el pedido debe estar en estado PENDIENTE
    And la mesa debe cambiar a estado OCUPADA

  Scenario: Procesar pago de un pedido
    Given que existe un pedido en estado ENTREGADO
    When el cajero registra el pago con método EFECTIVO
    Then el pago debe quedar en estado COMPLETADO
    And se debe generar una factura PDF
```

### pytest-html — Reportes Visuales de Pruebas

pytest-html genera reportes en formato HTML con el resultado de cada caso de prueba, incluyendo estadísticas generales (pruebas pasadas, fallidas, omitidas), duración de ejecución y detalles completos de los errores. Estos reportes se incluyen en el repositorio como evidencia del ciclo de pruebas.

```
# Instalación
pip install pytest-html

# Generar reporte HTML completo
pytest tests/ -v --html=reports/test_report.html --self-contained-html

# Generar reporte de cobertura HTML
pytest tests/ --cov=. --cov-report=html:reports/coverage_report/
```

## 5. Diseño y Ejecución de Casos de Prueba

Se diseñaron y ejecutaron 20 casos de prueba distribuidos en tres niveles: unitaria, integración e interfaz. Todos los casos fueron ejecutados exitosamente (✓ PASS).

### 5.1 Pruebas Unitarias — Modelos y Validadores

| ID | Caso de Prueba | Entrada / Acción | Resultado Esperado | Estado |
|---|---|---|---|---|
| UT-01 | Cálculo de propina sugerida | subtotal = 100.00 | propina = 10.00 (10%) | ✓ PASS |
| UT-02 | Validación de email correcto | email = 'user@mail.com' | Retorna True (válido) | ✓ PASS |
| UT-03 | Validación de email incorrecto | email = 'usermail' | Retorna False (inválido) | ✓ PASS |
| UT-04 | Validación de teléfono válido | telefono = '0998765432' | Retorna True | ✓ PASS |

| | | | | |
|---|---|---|---|---|
| UT-05 | Validación de dinero negativo | monto = -5.00 | Retorna False (valor inválido) | ✓ PASS |
| UT-06 | Estado inicial de mesa nueva | Mesa(numero=10) | estado = LIBRE | ✓ PASS |
| UT-07 | División de cuenta entre 4 | total=80.00, personas=4 | por_persona = 20.00 | ✓ PASS |

## 5.2 Pruebas de Integración — Controladores y Base de Datos

| ID | Caso de Prueba | Entrada / Acción | Resultado Esperado | Estado |
|---|---|---|---|---|
| IT-01 | Crear cliente y asignar a mesa | Crear cliente con mesa_id=1 | Cliente guardado, mesa en OCUPADA | ✓ PASS |
| IT-02 | Registrar pedido completo | Pedido con 3 detalles (platos) | Pedido creado, detalles en BD, total=45.00 | ✓ PASS |
| IT-03 | Descuento de ingredientes al vender | Venta de plato con 3 ingredientes | Stock de ingredientes decrementado en BD | ✓ PASS |
| IT-04 | Procesar pago y cerrar mesa | Pago EFECTIVO de pedido_id=2 | Estado pedido=PAGADO, mesa=LIBRE, PDF OK | ✓ PASS |
| IT-05 | Registrar turno de empleado | Inicio turno empleado_id=3 | Turno creado con fecha_inicio registrada | ✓ PASS |
| IT-06 | Consulta JOIN pedidos-mesas | SELECT con JOIN mesa-pedido | Retorna lista con datos de ambas tablas | ✓ PASS |
| IT-07 | Rollback ante error en pago | Pago con monto=None (inválido) | BD sin cambios, mensaje de error retornado | ✓ PASS |

## 5.3 Pruebas de Interfaz — Navegación y Flujos GUI

| ID | Caso de Prueba | Entrada / Acción | Resultado Esperado | Estado |
|---|---|---|---|---|
| UI-01 | Apertura de ventana principal | Ejecutar main.py | MainWindow visible con Sidebar activo | ✓ PASS |
| UI-02 | Navegación entre módulos | Click en cada ítem del Sidebar | Cada página carga sin errores ni excepciones | ✓ PASS |
| UI-03 | Formulario de nuevo cliente | Ingresar datos y click Guardar | Registro en BD, mensaje de confirmación OK | ✓ PASS |
| UI-04 | Mensaje de error campo vacío | Click Guardar con campos vacíos | QMessageBox de error visible al usuario | ✓ PASS |
| UI-05 | Reporte gráfico generado | Click en Reportes → Platos vendidos | Gráfico circular de Matplotlib renderizado | ✓ PASS |
| UI-06 | Generación de ticket PDF | Completar pago y click Generar PDF | Archivo PDF creado en directorio del sistema | ✓ PASS |

# 6. Reporte de Cobertura de Código

| Módulo / Archivo | Líneas Totales | Líneas Cubiertas | Cobertura (%) |
|---|---|---|---|
| models/pedidos.py | 62 | 57 | 92% |
| models/pagos.py | 48 | 45 | 94% |
| models/mesas.py | 38 | 36 | 95% |
| models/clientes.py | 41 | 38 | 93% |
| models/ingredientes.py | 44 | 40 | 91% |
| controllers/pedidos_controller.py | 70 | 63 | 90% |
| controllers/pagos_controller.py | 55 | 52 | 95% |
| controllers/mesas_controller.py | 42 | 38 | 90% |
| utils/validators.py | 35 | 35 | 100% |
| database/db_manager.py | 28 | 26 | 93% |
| TOTAL | 463 | 430 | 93% |

Cobertura total obtenida: 93% — Supera el mínimo requerido del 80%. Los módulos de validadores (validators.py) alcanzaron el 100% de cobertura.

# 7. Repositorio de GitHub — Entregable

El repositorio contiene: código fuente actualizado, instalador ejecutable, scripts de prueba, reportes de cobertura y documentación completa del proyecto.

## 7.1 Link del Repositorio

**Repositorio GitHub:** https://github.com/Alexander200318/restaurante_mvc_pyqt6.git

## 7.2 Contenido del Repositorio

| Carpeta / Archivo | Descripción |
|---|---|
| src/ | Código fuente completo del sistema (MVC: models, views, controllers, database) |
| installer/RestauranteApp.spec | Archivo de configuración de PyInstaller para generar el ejecutable |

| | |
|---|---|
| installer/RestauranteApp_Setup.iss | Script de Inno Setup para crear el instalador gráfico |
| installer/RestauranteApp_Installer_v1.0.exe | Instalador ejecutable final listo para distribución en Windows |
| tests/test_models.py | Pruebas unitarias de los modelos de dominio del sistema |
| tests/test_controllers.py | Pruebas unitarias de los controladores (lógica de negocio) |
| tests/test_database.py | Pruebas unitarias del gestor de base de datos y ORM |
| tests/test_integration.py | Pruebas de integración (controladores ↔ base de datos) |
| reports/test_report.html | Reporte HTML de resultados de todas las pruebas ejecutadas |
| reports/coverage_report/ | Reporte HTML detallado de cobertura de código (93%) |
| requirements.txt | Lista de dependencias del proyecto incluyendo librerías de prueba |
| requirements-dev.txt | Dependencias de desarrollo: pytest, pytest-cov, pytest-mock, behave |
| README.md | Documentación completa del proyecto con instrucciones de instalación |

### 7.3 Instrucciones de Instalación desde el Repositorio

6. **Descargar el instalador:** Ir a la carpeta installer/ del repositorio y descargar RestauranteApp_Installer_v1.0.exe.
7. **Ejecutar el instalador:** Doble clic sobre el archivo .exe y seguir el asistente de instalación (seleccionar directorio, crear accesos directos).
8. **Iniciar la aplicación:** Abrir el acceso directo creado en el Escritorio o desde el menú Inicio de Windows.
9. **Instalación alternativa:** Clonar el repositorio, crear entorno virtual, instalar requirements.txt y ejecutar python main.py.

## 8. Dependencias de Pruebas (requirements-dev.txt)

El archivo requirements-dev.txt contiene todas las librerías necesarias para ejecutar el ciclo de pruebas en el entorno de desarrollo:

```
# requirements-dev.txt
# Librerías para el Ciclo de Pruebas — Sistema de Restaurante
```

```
# —— Framework principal de pruebas ——
pytest==8.3.2
pytest-cov==5.0.0
pytest-mock==3.14.0
pytest-html==4.1.1

# —— Cobertura de código ——
coverage==7.6.1

# —— Pruebas de comportamiento (BDD) ——
behave==1.2.6

# —— Mocking y fixtures ——
factory-boy==3.3.0
Faker==26.0.0

# —— Instalación ——
# pip install -r requirements-dev.txt

# —— Ejecución completa de pruebas ——
# pytest tests/ -v --cov=. --cov-report=html --html=reports/test_report.html
```

## 9. Conclusión

El desarrollo del instalador y la ejecución del ciclo de pruebas del Sistema de Gestión de Restaurante se completaron satisfactoriamente. El instalador generado con PyInstaller e Inno Setup permite distribuir la aplicación en entornos Windows sin necesidad de instalar Python ni ninguna dependencia adicional.

El ciclo de pruebas implementado con pytest, unittest, pytest-cov y las librerías complementarias permitió validar el correcto funcionamiento de todas las capas del sistema: modelos de dominio, controladores de lógica de negocio e integración con la base de datos. La cobertura de código del 93% supera ampliamente el estándar requerido del 80%.

Todos los entregables (instalador, scripts de prueba, reportes de cobertura y documentación actualizada) han sido subidos al repositorio de GitHub del proyecto, quedando disponibles para su revisión y descarga.

## 10. Referencias

**Repositorio GitHub:** https://github.com/Alexander200318/restaurante_mvc_pyqt6.git
**pytest Documentation:** https://docs.pytest.org/
**PyInstaller Documentation:** https://pyinstaller.org/
**Inno Setup Documentation:** https://jrsoftware.org/isinfo.php
**coverage.py Documentation:** https://coverage.readthedocs.io/